

Solution for Forgetful BFS

Group the queue into **levels**:

- level 0 is [start]
- for each d , level $d + 1$ is obtained by walking level d from left to right and appending, for each element x , the whole list `neighbors(x)`

We can observe that level d can be seen as the sequence of endpoints of **all walks of length d starting at node 0**, listed in lexicographic order of the walks (as sequences of node numbers).

Thus, S_d , the size of level d , is simply the number of walks of length d from node 0. In general, this value grows exponentially, but we will never build the set explicitly, we'll only count its size.

Additionally, we can observe that node v first appears in level $\text{dist}(0, v)$, since the shortest walk to v has that length. So v is marked visited while level $\text{dist}(0, v) - 1$ is being popped from the queue.

Locating the last write

Let D be largest distance from 0 to a reachable node. (This is called the *eccentricity* of node 0.)

If $D = 0$, the only reachable node is 0 itself, and that node is marked as visited before we take any steps.

Otherwise the last update to the set of visited nodes must be when we mark some node at distance D , and that happens while level $D - 1$ is being popped. Since `steps` is incremented before each pop, the answer we seek is the sum of the sizes $S_0 + \dots + S_{D-2}$, plus the 1-based position within level $D - 1$ of the node for which we'll perform that final update.

A node t at distance D is marked as visited the first time any of its neighbors is popped while processing level $D - 1$. Thus, we only care about those neighbors of t who are at distance exactly $D - 1$ from the start.

The last update can therefore be found by iterating over all nodes t that are at the maximum distance D , for each of them iterating over their possible BFS parents (i.e., neighbors at distance $D - 1$) and for each of those determining the first appearance in level $D - 1$.

The order of first occurrences is simply the plain BFS order

Consider where the first occurrence of a node v in level d comes from. If u is a parent of v whose first occurrence in level $d - 1$ is at position p , then the copy of v that u spawns lands at position $\text{Prefix}(p) + \text{idx}_u(v)$, where $\text{Prefix}(p)$ is the total degree of the first p elements of level $d - 1$ and $\text{idx}_u(v)$ is v 's rank among u 's sorted neighbors. Since every reachable node has degree ≥ 1 , Prefix is strictly increasing, so minimizing over parents means: pick the parent whose first occurrence in level $d - 1$ is earliest, and among nodes of the same depth order by (position of that parent, node number).

That is precisely the order in which an ordinary BFS — one that scans each neighbor list once, in sorted order — discovers nodes. Hence **first-occurrence order at every level equals BFS discovery order**, and the best parent of each node is its ordinary BFS parent.

Thus, we can now tell more precisely when the last change happens:

- For each of the distance- D nodes we take their parent in a classical BFS.
- In this set of nodes, we want the one that appears last in level $D - 1$, and that is the one who's discovered last in the classical BFS order.

Note that all of the above arguments just referred to relative orders. We never needed to actually compute or even compare any big integer counts.

Evaluating the two counts

Let $y_k[u]$ be the number of walks of length k starting at u . These values can be computed efficiently using dynamic programming. (Algebraically, $y_0 = \mathbf{1}$ and $y_{k+1} = A y_k$ where A is the adjacency matrix.)

Once we know these, we also know all $S_d = y_d[0]$.

The first occurrence of the node v we identified corresponds to the lexicographically smallest walk $0 = w_0, w_1, \dots, w_{D-1} = v$, which is just the BFS parent chain of v reversed. The walks that precede it are exactly those agreeing with it on the first i steps and then stepping to a *smaller-numbered* neighbor of w_i , after which they may continue arbitrarily for the remaining number of levels.

Once we made these observations, we know everything we need to calculate the full answer: in one cycle we'll add up S_0 to S_{D-2} and in another cycle we'll go along the path from the start to v and sum up the y values that correspond to earlier neighbors + a corresponding number of steps.